

KIX

A Structured Specification for
Lightning Payment Infrastructures

WHITE PAPER

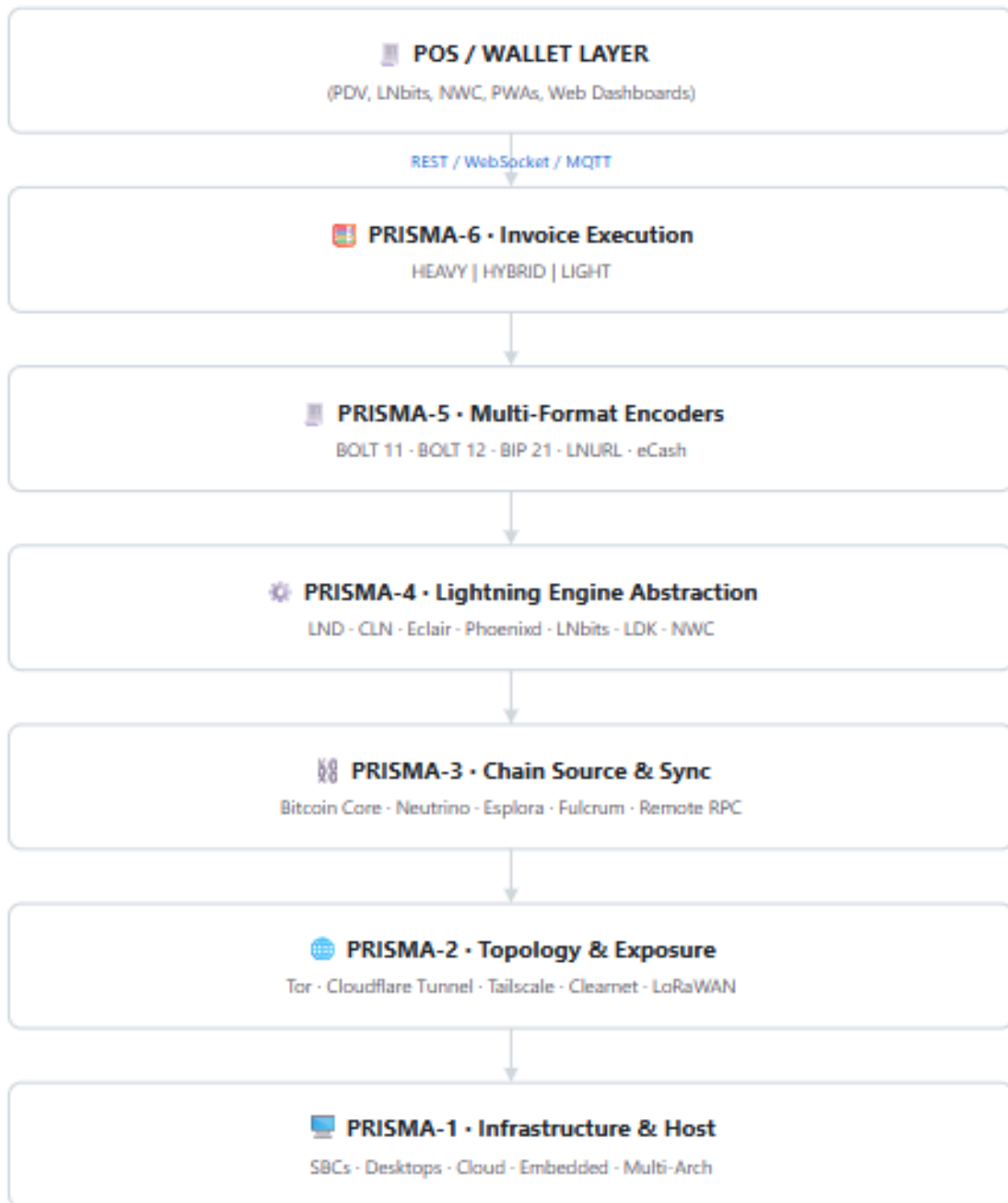
Technical Specification

Author: Hexarcs

KIX Protocol · P.R.I.S.M.A. Architecture

KIX Architecture Overview

P.R.I.S.M.A. layers from the POS / wallet interface to the infrastructure host.



⚡ KIX — Sovereign Lightning Infrastructure, Modular by Design ⚡

Figure 1. Layered view of the KIX payment infrastructure and its six PRISMA modules.

Abstract

The complexity of implementation and communication in second-layer payment networks often limits their large-scale adoption. We propose KIX, an open-source specification protocol designed to simplify cognitive load by fragmenting the Lightning Network architecture into modular and standardized stages. Instead of requiring each operator to exhaustively describe their own hardware and software stack to interact with the ecosystem, the protocol standardizes technical communication, allowing the community to refer to universal architectural layers. The model, grounded in the **KIX Core**, is strictly focused on the practical implementation of the network, offering users the flexibility to abstract themselves from the underlying technical complexity, should they wish to do so. As a specification based on open-source elements, KIX aligns the structural incentives necessary to scale payment infrastructure in a sovereign, decentralized, and non-custodial manner.

1. Introduction

The sovereign operation of a full Bitcoin node, coupled with traditional Lightning Network implementations — such as CLN, LND, or Eclair — often represents an intimidating technical barrier for local merchants and average individuals. Over the years, the community has focused on developing various technologies and abstractions to reduce this initial friction, with the constant goal of mitigating complexity and drastically lowering the computational requirements needed to use the payment network on a daily basis.

Despite these structural advances, it is observed that even software developers and engineers immersed in the Lightning ecosystem are often unaware of the vast diversity of hardware that can be employed in the practical implementation of the network. The architectural standardization proposed by the KIX protocol actively facilitates ecosystem development by creating a common technical language, allowing the community to understand operational capabilities ranging from corporate cloud servers to extremely low-cost, low-power microcontrollers.

Furthermore, the specification plays a crucial role in preventing a "tunnel vision" among implementers regarding communication topologies. KIX makes explicit that various exposure and network approaches can be adopted — and even executed in parallel — precisely defining at which stage of a node's architecture the system can be horizontally scaled using containerization tools such as Docker and its derivatives.

Finally, the protocol's organization architecturally locates the payment software ecosystem. It determines exactly where point-of-sale (POS) front-end solutions and wallet management ecosystems — such as LNbits or interfaces via NWC (Nostr Wallet Connect) — position themselves. In essence, the specification maps how higher-level modules communicate via API requests with the lower engines of the stack, ensuring that heavy processing and invoice routing are abstracted and delegated to the appropriate layers, freeing the end user from the underlying cryptographic and computational complexity.

2. PRISMA-1: Infrastructure Execution and Host Profiles

PRISMA-1 establishes the physical and operational foundations of the KIX protocol, defining the guidelines and specification of hardware environments and operating systems (supporting Linux, Android, and Windows environments) capable of supporting an implementation in accordance with the P.R.I.S.M.A. architecture. The choice of operating system and execution platform profoundly impacts the behavior of the other protocol modules, defining everything from the availability of native Lightning routing engines to network exposure and storage methods. The fundamental objective of this module is to provide a sovereign reference model for the execution layer.

2.1 Architectures and Instruction Requirements

The model guides the use of 64-bit environments to ensure toolchain interoperability. Although the philosophy is multi-architecture (x86_64, ARM64, and eventually RISC-V), the 64-bit standard is fundamental to maintaining the stability of critical components.

2.2 Sizing and Resource Impact

- **Restricted / Embedded Profiles (ARM64 / SBCs):** Devices such as NanoPi or Orange Pi operate under severe constraints (1 GB to 4 GB of RAM), requiring lightweight synchronization strategies.
- **Expanded Profiles (x86_64 / Servers and Cloud):** Local or cloud environments with greater capacity, allowing concurrent execution of full Lightning engines, indexers, and back-office.

2.3 Modular Composition by S.H.A.R.D.s (Software Hardware Automated Routing Division)

An Infrastructure S.H.A.R.D. represents the basic physical and operational unit of the host environment. Examples:

- **Micro-Embedded:** Android terminal (Sunmi V2) or ultra-compact SBC. Focused on API calls via remote connection.
- **Edge / Shop Floor:** SBC (NanoPi, Raspberry Pi 4) for local counter server.
- **Standalone Desktop:** Dedicated PC/Mini-PC for running a full local node and POS microservices.
- **Cloud / Enterprise Node:** Virtualized instance acting as a centralizing server (Cluster/Multi-Tenant).

3. PRISMA-2: Topology, Deployment, and Network Exposure

PRISMA-2 defines the guidelines for containerization, process isolation, and network exposure strategies. The objective is to standardize how KIX microservices are packaged, orchestrated, and made available, covering everything from containerization environments (Docker, Podman) to direct execution (bare-metal).

3.1 Modular Composition by Deployment and Exposure S.H.A.R.D.s

- **Sovereign Darknet:** Tor .onion v3 addresses isolated in containers, focused on total sovereignty and privacy.
- **Secure Tunnel:** Dynamic orchestration exposing services via Cloudflare Tunnel or mesh VPN networks (Tailscale) with low latency.
- **Public Clearnet:** Native or host-network execution exposed via direct IPv4/IPv6 with reverse proxy (Nginx/Caddy) and automatic TLS.
- **Mesh & Offline-Resilient:** Transmission via LoRaWAN radio or local p2p networks (Wi-Fi/Bluetooth) for locations without internet coverage.

4. PRISMA-3: Chain Source and Backend Synchronization

PRISMA-3 specifies the guidelines for connection, state validation, and synchronization with the Bitcoin blockchain, ensuring that the application determines the real state of the network with sovereignty, supplying the settlement engines.

4.1 Synchronization Mechanisms and Data Sources

- **Sovereign Full:** Direct execution of Bitcoin Core (Full/Pruned Node) with complete local validation (Zero-Trust) coupled via ZMQ and RPC.
- **Compact Light:** Light clients via compact filters (BIP 157/158), such as Neutrino, with extreme storage efficiency.
- **Indexed Gate:** Intermediate layers (Esplora / Fulcrum) for instant queries and mempool support.
- **Remote RPC / Provider Relay:** Middleware connecting the POS to remote nodes managed by third parties, focusing on low latency and no local synchronization.

5. PRISMA-4: Lightning Engine Abstraction

PRISMA-4 specifies the abstraction and switching layer between settlement engines, making POS interfaces entirely agnostic to the payment backend. It allows dynamically switching between engines running locally or hosted by third parties.

5.1 Modular Composition by Lightning Engine S.H.A.R.D.s

- **Native Enterprise:** Direct engines such as LND or Core Lightning for high-volume sovereign nodes.
- **Modular Engine:** Local LNbits or Alby Hub, ideal for multi-tenant ecosystems and store isolation.
- **Autonomous Server:** Engines such as Phoenixd focused on automatic liquidity management via splicing.
- **Delegated Remote:** Remote wallets via Nostr Wallet Connect (NWC), connected as a client without local node infrastructure.
- **Embedded SDK:** API providers or cloud backends based on LDK (Lightning Development Kit) with keys held by the client.

6. PRISMA-5: Multi-Format Invoice Encoders

PRISMA-5 ensures maximum interoperability at the POS by dynamically generating and converting native network billing formats and proprietary contingency schemes.

6.1 Standardized Formats and Fallback Strategies

- **Native Standard:** Native generation of BOLT 11 invoices (single-use) and support for Offers (BOLT 12).
- **Unified Universal:** BIP 21 QR Codes grouping On-Chain and Lightning, integrated with LNURL-Pay resolution to cover any customer wallet.
- **Contingency & Tokenized:** Conversion to eCash Tokens (Cashu/Fedimint) or local credit vouchers during network interruptions. Issues contingent invoices with queued settlement for later compensation without halting the cash register.

7. PRISMA-6: Invoice Execution Profiles

PRISMA-6 specifies the distribution of computational load between the point-of-sale terminal (POS) and the back-office infrastructure, decoupling the terminal's logic from the need for high resources.

7.1 Computational Load Distribution

- **Heavy Profile (PRISMA-6-HEAVY):** Servers (x86_64/ARM64) running web frameworks (Django, FastAPI), managing databases, full Lightning nodes, and synchronous e-commerce webhooks.
- **Intermediate Profile (PRISMA-6-HYBRID):** Portable Android POS devices (Termux/Sunmi) capable of running PWAs and native applications, managing local certificates and printing receipts via cloud API.
- **Light Profile (PRISMA-6-LIGHT):** Microcontrollers (ESP32, STM32 with TFT displays). They act purely as a visual interface. They transmit the order value via MQTT, WebSockets, or REST to the HEAVY server, render the returned QR Code, and wait for the callback without processing cryptography locally.